

Curso de Ciência da Computação

Programação Orientada a Objetos – POO

Profa. Fernanda dos Santos Cunha

MATERIAL

LIVROS BIBLIOTECA DIGITAL

SCHILD, Herbert. Java para iniciantes. 6ª ed. 2015.

SILVA, Fabricio. Paradigmas de Programação. 2019. (Caps 5 a 9)

OBERLEITNER, Allen; MASIERO, Andrey Araujo. Programação orientada a objetos. Série Universitária: Senac. 2021.

LIVRO BIBLIOTECA FÍSICA

DEITEL, Paul J.; DEITEL, Harvey M. Java™: como programar. 10ª ed. 2016. (11ª ed. em inglês, 2017)

FERRAMENTAS

StarUML - <https://staruml.io/>

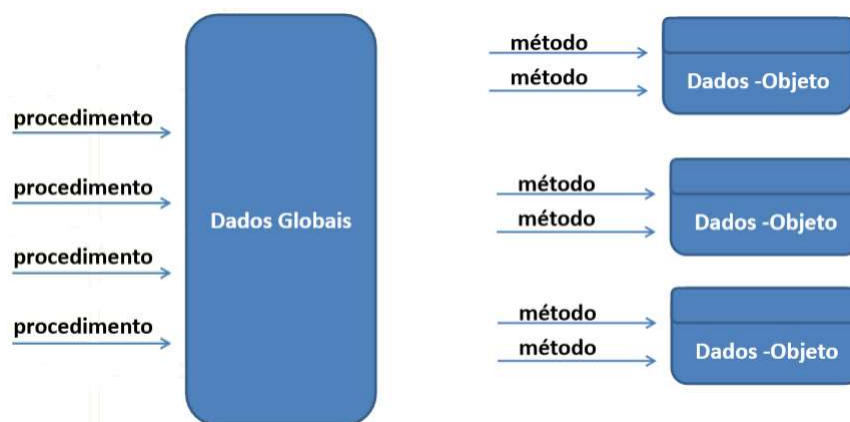
IntelliJ IDEA - <https://www.jetbrains.com/idea/>

Histórico POO



- Simula, década de 60, Noruega, baseada em ALGOL 60
 - simulações de sistemas com grande número de objetos
 - base para Tipos Abstratos de Dados (TAD)
 - base para SmallTalk, década de 70, Xerox
- Década de 80:
 - C++ e Objective-C, baseadas em C
 - Pascal orientado a objetos
 - Eiffel

Paradigma Estruturado x Paradigma Orientado a Objetos



Paradigma Estruturado x Paradigma Orientado a Objetos



- Mapeamento de problema do mundo real a ser resolvido num modelo computacional.
- Programação Estruturada
 - Consiste na criação de um conjunto de procedimentos (algoritmos) para resolver o problema
 - Encontrar modos apropriados de armazenar os dados
- Programação Orientada a Objetos
 - Consiste em identificar os objetos e as operações relevantes no mundo real
 - Mapeamento desses em representações abstratas no espaço de soluções

Paradigma Orientado a Objetos



- Paradigma de Programação dominante nos dias atuais.
- Substituiu as técnicas de programação procedimental (estruturada).
- “Fornece um mapeamento direto entre o mundo real e as unidades de organização utilizadas no projeto”.
- Diversas unidades de software, chamadas de objetos, interagem entre si.
- Separa claramente a noção de o que é feito de como é feito.

Paradigma Orientado a Objetos



- Aumento na reutilização de código.
- Sensível redução no custo de manutenção do software.
- A Programação Estruturada também pode apresentar estas características, tudo depende de como se programa, assim como também é válido dizer que não basta utilizar-se da POO para garantir os itens acima.

Vantagens do Paradigma Orientado a Objetos



- Unificação de dados e processos via classes (atributos x métodos).
- Consistência entre análise e desenvolvimento (relação direta entre as entidades conceituais e as entidades físicas a serem implementadas).
- Reutilização de código (herança, polimorfismo e associação).
- Multidesenvolvimento (o desenvolvedor apenas necessita conhecer as classes e os objetos desenvolvidos pelos colegas).
- Facilidade de manutenção (criar novas classes, métodos, atributos e relacionamentos facilmente para atender as novas demandas do sistema).

Unified Modeling Language UML



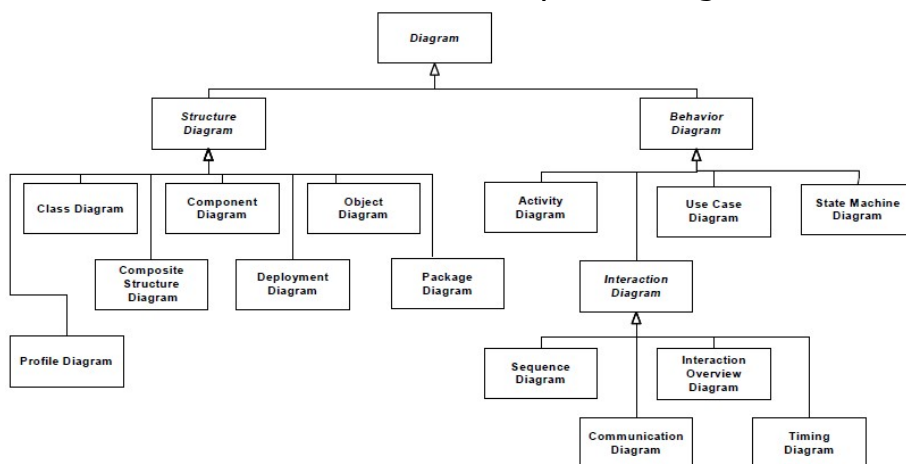
- Linguagem para especificar, construir, visualizar e documentar um sistema de software, surgida da fusão de metodologias anteriormente usadas (Grady Booch, Jim Rumbaugh e Ivar Jacobsen) – início em 1994 e aprovação pela OMG¹ em 1997.
- Linguagem para modelagem de sistemas orientados a objetos (principalmente).
- Modelagem visual para que relacionamentos entre os componentes do sistema sejam melhor compreendidos e documentados.
- A versão atual e oficial é a 2.5.1 (2017) – acesse <https://www.omg.org/spec/UML/2.5.1/About-UML>

1. Object Management Group®

UML



A UML 2.5.1 define 14 diferentes tipos de diagramas:



UML – Exemplo de Diagrama de classe

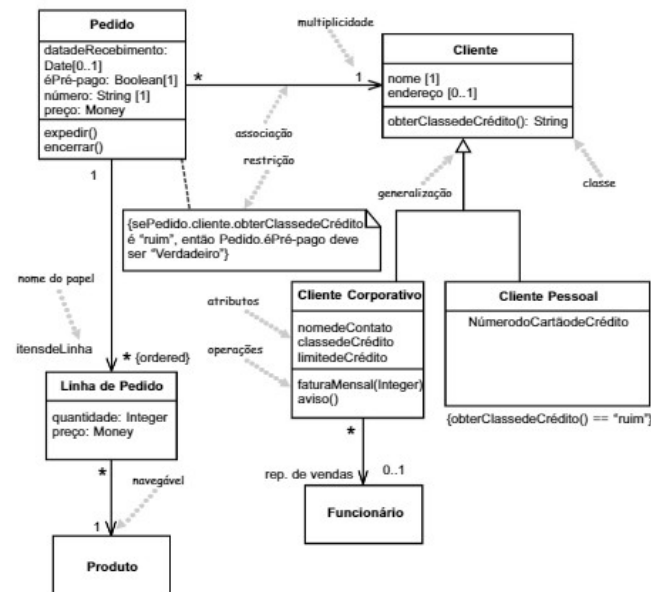


FIGURA 3.1 Um diagrama de classes simples.

Abstração



- Processo de **identificar os aspectos essenciais** de um contexto qualquer, ignorando características menos importantes ou acidentais.
- **Uma abstração** é o resultado deste processo.
- Não se analisa o "todo", em POO é importante **analisar as partes para entender o todo**.

Abstração



- Abstração é uma das formas fundamentais para lidar com a complexidade.
 - Quando se quer diminuir a complexidade de alguma coisa, ignora-se detalhes sobre as partes para concentrar a atenção no nível mais alto de um problema.

Abstração



- O que se vê ??? (BOOCH ET AL, 2007)



Abstração



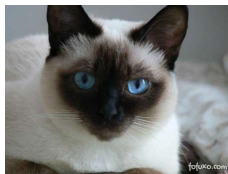
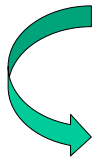
- Níveis de abstração

- Animal

- Mamífero

- Gato

- » Gato Siamês



(BOOCH ET AL, 2007)

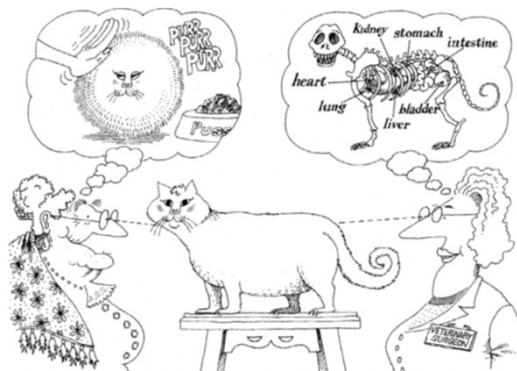
Visões diferentes



- Identificar os aspectos essenciais depende **do observador e do fenômeno observado.**

(BOOCH ET AL, 2007)

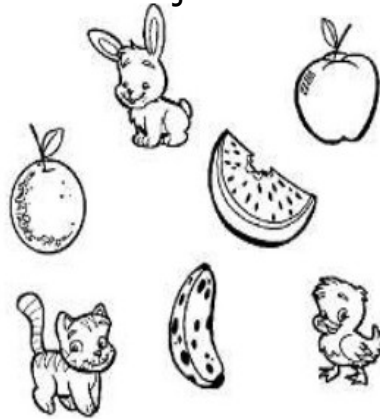
- Diferentes **observadores.**
- Diferentes **necessidades.**



Classificação



- Como classificar estas "coisas" ??
- Classificar é uma forma de abstração.



Classificação



ANIMAL



FRUTA

ABSTRAÇÕES



Classificação



- O que foi observado em cada “coisa”?
- Similaridades?
- Características comuns.
- Comportamento comum.
- Porém cada “coisa” é um indivíduo dentro do grupo.



Classe



- **Modelo** a partir do qual se criam objetos.
- Define **características** (atributos) e **comportamento** (operações/métodos) comuns a todos os objetos criados a partir dela.
- Deve representar bem uma única **abstração**.



Classes x objetos



- Objetos são criados a partir de uma classe.
- Cada objeto é uma instância de sua classe.
- Criação = instanciação.

Classe (Modelo/Molde)

Objetos criados a partir da classe



Objeto



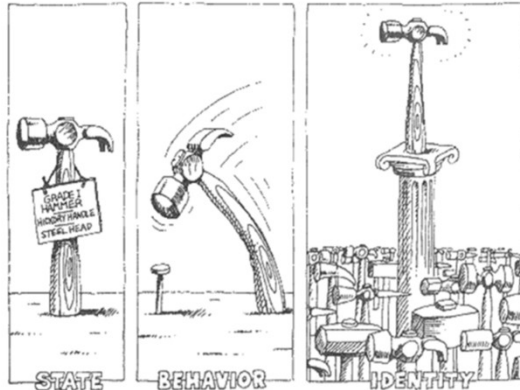
- Entidade única com identidade própria.
- Objetos de software modelam objetos reais.
- Estado de um objeto:
 - O **conjunto dos valores** dos seus atributos em um determinado momento representa o **estado do objeto**.

Objeto



- **Estado**
 - Martelo tipo 1
 - Cabo de nogueira
 - Cabeça de aço
- **Comportamento**
 - Martelar
- **Identidade**
 - Meu martelo

(BOOCH ET AL, 2007)



Atributos X Var. locais



- Um **atributo** é uma característica relevante para o objeto.
 - Não faz sentido falar do objeto sem esse atributo.
- Variáveis temporárias não devem ser declaradas como atributos.
 - Ex.: variável para controle de laço, para armazenar cálculos intermediários, etc.

Estado de um objeto



Objeto 1



Objeto 2



Classe
Automovel
numeroPortas
cor
fabricante
ano
placa

Atributos



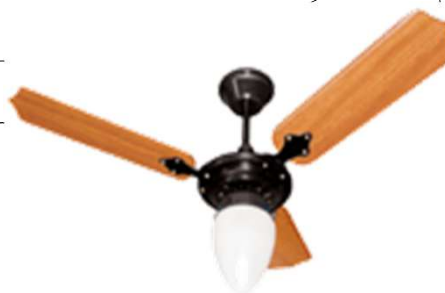
Estado de um objeto



- Objeto "ventilador 1"
- Número de pás: 3
- Material da pá: **MADEIRA**
- Número de velocidades: 3
- Cor: **MOGNO**
- Tem exaustor: **SIM**
- Tem lustre: **SIM**

Estado
atual

Atributos



Estado de um objeto



- Objeto **"ventilador 2"**
- Número de pás: **2**
- Material da pá: **PLÁSTICO**
- Número de velocidades: **4**
- Cor: **VERDE**
- Tem exaustor: **NÃO**
- Tem lustre: **SIM**



Comportamento de um objeto



- Como pedir ao objeto que ele faça alguma ação?
- Quais as ações sobre um ventilador?
 - Alterar número de pás.
 - Obter número de pás ?
 - Alterar cor.
 - Obter a cor ?
 - ...

Comportamento de um objeto



- Outras ações ?
 - Ligar / desligar modo ventilador.
 - Ligar / desligar modo exaustor.
 - Acender / desligar a luz.
 - Aumentar a velocidade.
 - ...

Comportamento de um objeto



- Mais ações ?
 - Está ligado ou não ?
 - Em qual modo está ligado ?
 - A luz está acesa ?
 - Qual a velocidade atual ?
 - ...

Comportamento de um objeto



- Como você identificou o comportamento ?
 - Qual a relação com a abstração ?
 - Qual seu ponto de vista?
 - Desenvolver um sistema de controle de estoque ?
 - Desenvolver um sistema de controle automático de ventiladores em um prédio inteligente ?
 - **O ponto de vista (necessidades) afeta a abstração !!**

Comportamento de um objeto



- O comportamento é representado por um conjunto de operações.
 - Elas são as responsabilidades da classe, determinando o que os objetos podem fazer (ações).

Identidade de um objeto



ventilador 1



ventilador 33

Objetos podem ter um mesmo estado...
Mas eles ainda são objetos diferentes!!

Instanciando e destruindo objetos

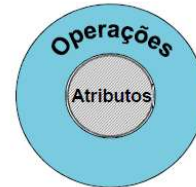


- Deve-se solicitar a classe que “instancie” um objeto.
 - **Construtor:** função que é chamada sempre que é criado um objeto de uma classe, faz a inicialização.
- O próprio objeto resolve quando deve ser “destruído”.
 - **Destrutor:** função que é chamada sempre que o escopo de duração do objeto de uma classe encerra-se - faz a “limpeza”.

Encapsulamento



- Mecanismo para agrupar os atributos e as operações que manipulam estes atributos.



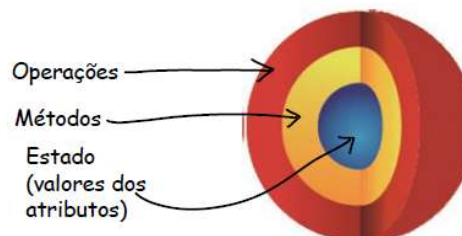
- **Esconder o estado** do objeto:
 - Ocultamento das informações.
 - A única forma de acessar ou modificar um objeto são as operações.
 - Proteção ao acesso descontrolado.



Encapsulamento



- **Esconder a implementação das operações** do objeto.
 - Método é a implementação de uma operação.



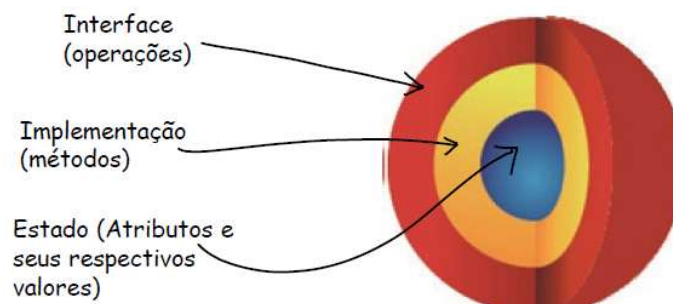
Ex.: a operação *imprimir* pode ser implementada de forma distinta, dependendo se o arquivo a ser impresso contém apenas texto ASCII, é um arquivo de um processador de texto ou binário.

Encapsulamento e Interface

- Toda classe implementa (realiza) uma **interface**.
- Interface define conjunto de operações visíveis (públicas) de uma classe.
 - Quais operações podem ser invocadas nos objetos de uma determinada classe.

Encapsulamento e Interface

- Encapsulamento permite separação entre a interface (operações) e sua implementação (métodos).



Nomear uma Classe



- O nome da classe deve estar relacionado com o **principal objetivo** dela.
- Qual abstração a classe representa ?
- **Princípio da Responsabilidade Única:**
 - Uma classe deve ter uma única responsabilidade, que deve ser totalmente encapsulada, e todas as operações devem estar fortemente alinhadas a esta responsabilidade.

Nomear uma Classe



- Utilizar um substantivo que represente a abstração.
- Expressar o nome no singular e adotar estilo de escrita `UpperCamelCase`².
- Utilizar a relação **é um/uma** do objeto para a classe: João **é um** Aluno.
- Ex.:
 - Professor, Aluno, Cliente, Produto...
 - NotaFiscal, ContaBancaria...

2. <http://pt.wikipedia.org/wiki/CamelCase>

Nomear uma Classe



- Coesão:
 - Medida originária do paradigma estruturado, relacionada a Modularidade, que indica o quanto os elementos de um módulo fazem sentidos juntos
- Métrica interna a classe.
- Manter ALTA COESÃO:
 - **Quanto maior a coesão, melhor foi aplicado o Princípio da Responsabilidade Única.**

Nomear um Atributo



- Utilizar nomes significativos, com relação ao problema.
- Mostrar claramente o que representa.
- Evitar mnemônicos e nomes muito longos.
- Adotar estilo de escrita `LowerCamelCase`³.
- Ex.:
 - `dataNascimento`, `nomeCompleto`, `enderecoResidencial`, ...

3. <http://pt.wikipedia.org/wiki/CamelCase>

Nomear uma Operação



- Utilizar o domínio do problema para identificar nomes significativos e adequados a ação realizada.
- Evitar mnemônicos e nomes muito longos.
- Adotar estilo de escrita `LowerCamelCase`.
- Evitar muitos parâmetros, mantendo legibilidade na **assinatura da operação**.
- Ex.:
 - `setEnderecoResidencial,`
`transmitirArquivoContas, getSaldo...`

Assinatura de uma Operação



- Nome da operação.
- Visibilidade (geralmente `public`, ou `protected`, `private`).
- Lista de parâmetros.
- Tipo do retorno.

```
public int fazAlgo(float nomeParametro)
```

Nomear uma Operação



- Cada operação deve realizar um serviço.
 - Dificuldade em nomear => existe mais de um serviço oferecido pela operação ?
 - Muitos parâmetros => muitos serviços?
 - Lembrar da **coesão!!**

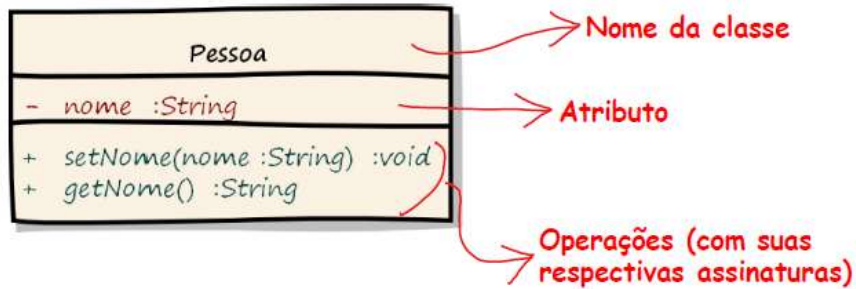
Exercício



- Quais características estão presentes em qualquer pessoa ??
- E se considerarmos o contexto de um sistema bancário.



Exemplo - UML



Exemplo - Java



Arquivo Pessoa.java

```
public class Pessoa{  
    private String nome;  
    // método para definir o nome do objeto  
    public void setNome(String nome){  
        this.nome = nome; // armazena no atributo  
    }  
    // método para recuperar o nome do objeto  
    public String getNome(){  
        return nome; // retorna o valor do atributo  
    }  
} // fim da classe
```



A palavra reservada **this**



- Dentro de uma operação de objeto ou construtor, **this** é uma referência ao objeto corrente (objeto chamador).
- Muitas vezes a razão mais comum para o uso do **this** é para diferenciar um atributo de um parâmetro de mesmo nome.
 - Ex.: construtor `Data` com parâmetros.
- O ponteiro **this** é implícito para todas as funções membro de uma classe.

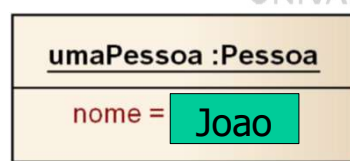
Instanciando Objetos



Arquivo PessoaTest.java

```
import java.util.Scanner;

public class PessoaTeste{
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);
        Pessoa umaPessoa = new Pessoa(); // cria objeto
        System.out.println("Nome:");
        String umNome = input.nextLine();
        umaPessoa.setNome(umNome);
        System.out.printf("Nome: %s\n", umaPessoa.getNome());
    }
}
```



Construtor

Pessoa();



Default: retorna
um objeto Pessoa



- Método de inicialização de uma classe, invocado automaticamente no momento da criação de objetos.
- Permite inicializar campos internos da classe e alocar recursos que um objeto da classe possa necessitar.
- Não pode retornar valor (nem mesmo `void`)
- Existe construtor padrão na classe que não declare explicitamente um construtor.
- Quando se declara um construtor, o construtor padrão não será criado.

Exemplo



Arquivo Pessoa.java

```
public class Pessoa{
    private String nome;
    // constructor com parâmetro
    public Pessoa(String nome){
        this.nome = nome; // string vazia
    }
    // método para definir o nome do objeto
    public void setNome(String nome){
        this.nome = nome; // armazena no atributo
    }
    // método para recuperar o nome do objeto
    public String getNome(){
        return nome; // retorna o valor do atributo
    }
} // fim da classe
```

Instanciando Objetos

Arquivo PessoaTest.java



```
public class PessoaTeste{
    public static void main(String[] args){
        Pessoa pessoa1 = new Pessoa("Joao"); // cria objeto
        Pessoa pessoa2 = new Pessoa("Maria");
        System.out.printf("Nome1:%s\n",pessoa1.getNome());
        System.out.printf("Nome2:%s\n",pessoa2.getNome());
    }
}
```

Destrutor

~Pessoa();
↓
Default: "destroi" o objeto



- Realiza a função inversa do construtor, pois é um método invocado automaticamente quando o objeto está para "morrer".
- Caso um objeto tenha recursos alocados, destrutores devem liberar tais recursos.
- **Em Java não se tem destrutor:** automaticamente o Garbage Collector varre a memória e libera o espaço de objetos que não estejam mais sendo referenciados.

```

public class Account {
    private String name;    // variavel de instancia
    private double balance; // saldo
    // construtor
    public Account( String name, double initialBalance ) {
        this.name = name;
        // valida initialBalance>0.0, se nao mantém valor padrao 0.0
        if ( initialBalance > 0.0 )
            balance = initialBalance;
    }
    // deposito de quantia valida no saldo
    public void deposit( double amount ){
        if ( amount > 0.0 )
            balance = balance + amount; // adiciona quantia ao saldo
    }
    // retorna saldo
    public double getBalance(){
        return balance;
    }
    // define nome
    public void setName(String name){
        this.name = name;
    }
    // retorna nome
    public String getName(){
        return name;
    }
} // end class Account

```



Outro exemplo
(Account.java)

```

package br.univali.poo;

import java.util.Scanner;

public class AccountTest {
    // main method begins execution of Java application

    public static void main( String[] args ) {
        Account account1 = new Account("Jane Green", 50.00 ); // cria objeto
        Account account2 = new Account("John Blue", -7.53 ); // cria objeto

        // exibe saldo inicial objetos
        System.out.printf("%s balance: %.2f\n", account1.getName(),
            account1.getBalance());
        System.out.printf("%s balance: %.2f\n\n", account2.getName(),
            account2.getBalance());

        // cria Scanner para obter entrada de dados
        Scanner input = new Scanner( System.in );

        // faz 1a entrada
        System.out.println("Enter deposit amount for account1: "); // prompt
        double depositAmount = input.nextDouble(); // obtem input
        System.out.printf("adding %.2f to account1 balance", depositAmount);
        account1.deposit( depositAmount ); // deposita quantia
        ...
    }
}

```

Outro exemplo
(AccountTest.java)



Outro exemplo

(AccountTest.java)



```
...
// exhibe saldos
System.out.printf("%s balance: $%.2f%n", account1.getName(),
    account1.getBalance());
System.out.printf("%s balance: $%.2f%n", account2.getName(),
    account2.getBalance());

// faz 2a entrada
System.out.print("Enter deposit amount for account2: "); // prompt
depositAmount = input.nextDouble();
System.out.printf("adding %.2f to account2 balance", depositAmount);
System.out.println();
account2.deposit(depositAmount ); // add to account2 balance

// exhibe saldos
System.out.printf("%s balance: $%.2f%n", account1.getName(),
    account1.getBalance());
System.out.printf("%s balance: $%.2f%n", account2.getName(),
    account2.getBalance());

} // fim main
} // fim classe AccountTest
```

Outro exemplo



```
AccountTest x
"C:\Program Files\Java\jdk-11.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetB
Jane Green balance: $50,00
John Blue balance: $0,00

Enter deposit amount for account1:
25,53
adding 25,53 to account1 balance

Jane Green balance: $75,53
John Blue balance: $0,00

Enter deposit amount for account2: 123,45
adding 123,45 to account2 balance

Jane Green balance: $75,53
John Blue balance: $123,45

Process finished with exit code 0
```

LEITURA PARA CASA



LIVROS

Schildt, H. Java para iniciantes - Caps 1 a 3: Fundamentos de Java, tipos de dados e instruções de controle; e Cap 4: Introdução a classes, objetos e métodos

Silva, F. Paradigmas de Programação - Cap 6: Classes, relacionamentos e encapsulamentos